11. CI/CD

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 11 of 18 · Thirupathi Keloth · v2.0

Contents

11.0 Why CI/CD

11.1 Core concepts

11.2 Anatomy of a pipeline

11.3 Versioning, artifacts and supply chain

11.4 Environments, configuration and promotion

11.5 Deployment strategies in depth

11.6 GitHub Actions in depth

11.7 Jenkins in depth

11.8 Other CI/CD platforms

11.9 Quality gates and testing in pipelines

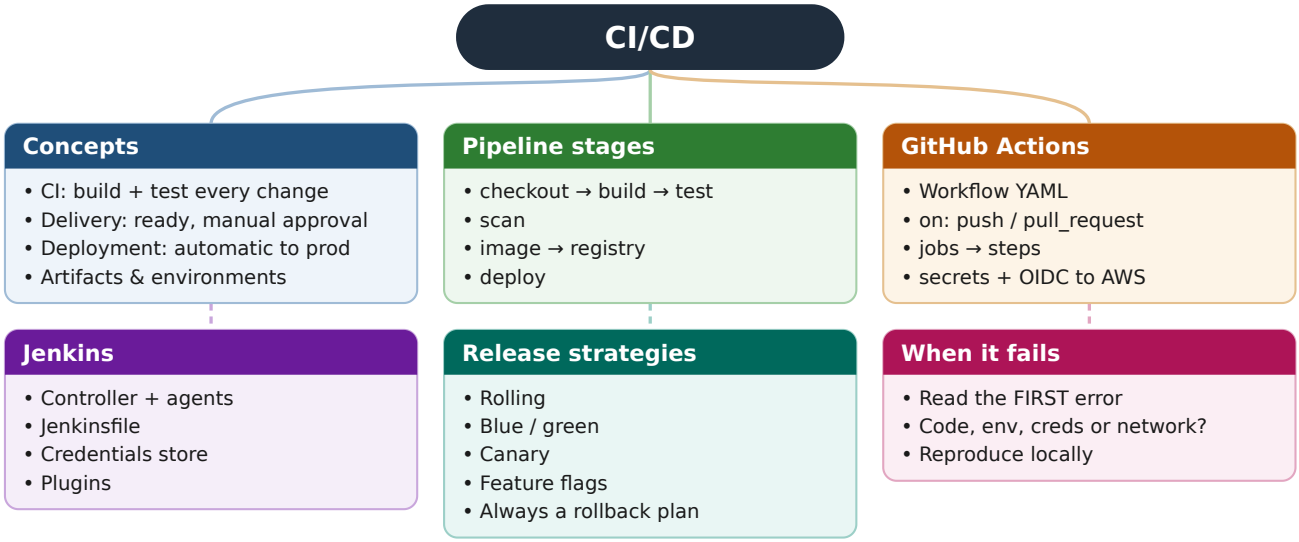
11.10 Securing the pipeline (DevSecOps in CI/CD)

11.11 Troubleshooting pipelines

11.12 Labs

11.13 Interview questions and answers

11.14 Self-check: I can...



11.0 Why CI/CD

Without automation, releases are big, risky, manual events: "merge everything on Friday, build on someone's laptop, copy files to servers, hope". CI/CD turns delivery into a **repeatable, automated, fast and safe** pipeline: every change is built, tested, scanned and deployable within minutes. It is the core of DevOps work.

11.1 Core concepts

Term	Meaning
Continuous Integration (CI)	Developers merge small changes to the main branch frequently; every change triggers an automatic build and tests, so integration problems are found within minutes
Continuous Delivery	Every change that passes the pipeline is **releasable**; deploying to production is a business decision (one click / approval)

Term	Meaning
Continuous Deployment	Every passing change goes to production **automatically**, no human gate
Pipeline	Automated sequence of stages defined as code (`.github/workflows/*.yml` , `Jenkinsfile` , `.gitlab-ci.yml`)
Stage / job / step	Phase (build, test, deploy) / unit of work on one runner / single command or action
Runner / agent	Machine/container that executes jobs
Artifact	Output of a build kept for later stages (container image, JAR, zip, test reports)
Environment	Deployment target (dev, staging, prod) with its own config, secrets and protection rules
Trigger	What starts a pipeline: push, pull request, tag, schedule, manual, another pipeline
Quality gate	Condition that must pass to continue (tests, coverage, no critical vulnerabilities, approval)

```
CONTINUOUS INTEGRATION                    CONTINUOUS DELIVERY / DEPLOYMENT
commit → build → unit tests → lint →     publish artifact → deploy dev → integration/e2e tests →
SAST/deps scan → package (image) →       deploy staging → (manual approval = Delivery) → deploy prod
                                          → verify (smoke tests, metrics) → (auto = Deployment)
```

Principles

1. **Everything in version control:** app code, pipeline definitions, infrastructure, configuration.
2. **Small, frequent changes:** smaller blast radius, easier reviews and rollbacks.
3. **Fast feedback:** CI on a PR should finish in minutes (parallelism, caching, test selection).
4. **Build once, deploy many:** one immutable artifact (image digest) promoted through all environments; only configuration differs.
5. **Fail fast:** cheap checks first (lint, unit tests) before expensive ones (e2e, deployments).
6. **Keep main green:** a broken main branch is everyone's top priority.
7. **Automate rollback** and make deployments boring.
8. **Shift left:** test and scan for security early, in the PR.

DORA metrics (how teams measure delivery performance)

Metric	Measures	Elite teams (indicative)
Deployment frequency	How often you deploy to production	On demand (many per day)
Lead time for changes	Commit → running in production	Less than a day
Change failure rate	% of deployments causing incidents/rollbacks	Low (around 0–15%)
Time to restore service (MTTR)	How quickly you recover from failures	Less than an hour

11.2 Anatomy of a pipeline

Stage	Typical steps	Fails when
Source	Checkout, determine version (git SHA/tag)	—
Static checks	Formatting, lint (ruff, eslint, hadolint, tflint), type checks	Style/lint errors

Stage	Typical steps	Fails when
Build	Compile, install dependencies (with cache)	Compilation/dependency errors
Unit tests	Fast isolated tests + coverage report	Test failures, coverage below threshold
Security (shift-left)	Secret scan (gitleaks), SAST (CodeQL/Semgrep), dependency scan (SCA), IaC scan	Leaked secrets, critical findings
Package	Build container image / artifact, SBOM, sign	Build errors
Image scan	Trivy/Grype on the image	Critical/high CVEs
Publish	Push to registry/artifact repo with immutable tags	Auth/registry errors
Deploy to dev/test	Helm/kubectl/ECS/Terraform/GitOps update	Rollout fails, health checks fail
Integration / e2e / contract tests	Tests against deployed services	Broken integrations
Approval (Delivery)	Human gate for production	Rejected
Deploy to prod	Progressive strategy (rolling/canary/blue-green)	Health/metrics checks fail → rollback
Verify	Smoke tests, synthetic checks, error-rate/latency analysis	Regressions
Notify	Chat/email, update tickets, release notes	—

Test types (the testing pyramid)

```
      /\        End-to-end / UI tests      few, slow, brittle, most realistic
     /  \       Integration / contract     some: services + DB/APIs together
    /    \      Unit tests                 many, fast, isolated
   /______\
```

Plus: **smoke tests** (quick "is it alive" after deploy), **performance/load tests** (k6, JMeter, Locust), **security tests** (DAST: OWASP ZAP), **chaos tests**.

Caches vs artifacts

	Cache	Artifact
Purpose	Speed up builds (dependencies, build cache)	Pass outputs between jobs / keep results
Correctness	Must be safe to lose	Part of the pipeline's result
Examples	`~/.cache/pip`, `node_modules` keyed by lockfile hash, Docker layer cache	Images, binaries, test/coverage reports, SBOMs, Terraform plans

11.3 Versioning, artifacts and supply chain

Topic	Practice
Version scheme	Semantic versions for releases (`v1.4.2` tag) + git SHA for every build (`sha-3f9a1c2`)
Artifact repositories	Container registries (ECR, GHCR, Docker Hub, Harbor), package repos (Nexus, Artifactory, CodeArtifact, PyPI/npm private), S3 for files
Immutability	A version is never overwritten (registry tag immutability); deploy by **digest** for certainty

Topic	Practice
Traceability	Labels/annotations linking artifact → commit → pipeline run → PR
SBOM	Software bill of materials generated at build (Syft, Trivy)
Signing and provenance	Sign images (cosign/Sigstore), produce build provenance (**SLSA** levels, GitHub artifact attestations); verify before deploy
Retention	Lifecycle policies to delete old/untagged artifacts

11.4 Environments, configuration and promotion

```
          same image digest sha256:ab12...
build ──► DEV (auto) ──► STAGING (auto + e2e) ──► PROD (approval / automated checks)
           config: dev     config: staging          config: prod (secrets from a secret manager)
```

Practice	Why
Promote artifacts, don't rebuild per environment	What you tested is exactly what you ship
Config outside the artifact (env vars, ConfigMaps, parameter stores)	Same image everywhere
Environment-specific secrets in the CI secret store / secret manager, scoped per environment	Dev pipelines can't touch prod secrets
Protection rules (required reviewers, wait timers, branch restrictions) on prod	Controlled releases
Production-like staging	Find problems before prod
Ephemeral preview environments per PR	Test features in isolation

Database migrations

Schema changes are the hardest part of continuous delivery. Use migration tools (Flyway, Liquibase, Alembic, Django/Rails migrations) run by the pipeline, and the **expand → migrate → contract** pattern: (1) add new columns/tables in a backward-compatible way, (2) deploy code using both, (3) backfill data, (4) remove old columns in a later release. This keeps every deployment **rollback-safe**.

Feature flags

Deploy code "dark" and switch features on per user/percentage at runtime (LaunchDarkly, Unleash, Flagsmith, AWS AppConfig). Separates **deploy** from **release**, enables trunk-based development and instant "rollback" by turning a flag off.

11.5 Deployment strategies in depth

```
ROLLING          v1 v1 v1 v1 → v2 v1 v1 v1 → v2 v2 v1 v1 → v2 v2 v2 v2
BLUE/GREEN       [blue v1 ← 100% traffic]   [green v2 ready] → switch → [green v2 ← 100%]
CANARY           v1 95% / v2 5% → watch metrics → 25% → 50% → 100% (or roll back)
```

Strategy	How	Downtime	Rollback	Extra cost	Risk
Recreate	Stop all old, start new	Yes	Redeploy old	None	High

Strategy	How	Downtime	Rollback	Extra cost	Risk
Rolling	Replace instances in batches (`maxSurge` / `maxUnavailable`)	No	Roll back gradually	Small	Medium: mixed versions during rollout
Blue/green	Full new environment, switch traffic (LB/DNS)	No	Instant switch back	2× during release	Low
Canary	Small % of traffic first, analyze metrics, increase	No	Shift traffic back	Small	Lowest (limited blast radius)
A/B testing	Route specific users to versions to compare business metrics	No	Route back	Small	Product experiment
Shadow (mirroring)	Copy real traffic to new version, discard responses	No	n/a	Extra capacity	Tests with real traffic safely
Feature flags	Code deployed off, enabled gradually	No	Turn flag off	Flag platform	Low

Tools: Kubernetes Deployments (rolling), **Argo Rollouts** / Flagger (canary, blue/green with automated metric analysis), AWS CodeDeploy (EC2/ECS/Lambda blue/green and canary), ALB weighted target groups, Route 53 weighted records, service meshes (Istio traffic splitting).

Automated verification: after each step, check error rate, latency (p95/p99), saturation and business KPIs; roll back automatically when thresholds are breached.

11.6 GitHub Actions in depth

Building blocks

Concept	Meaning
Workflow	YAML file in `.github/workflows/`; a repo can have many
Event (`on`)	Trigger: `push`, `pull_request`, `workflow_dispatch` (manual, with inputs), `schedule` (cron, UTC), `release`, `workflow_call` (reusable), `workflow_run`, `merge_group`, `repository_dispatch`
Job	Group of steps on one runner; jobs run **in parallel** unless linked with `needs`
Step	`run:` a shell command or `uses:` an action
Action	Reusable unit (`actions/checkout@v4`, `docker/build-push-action@v6`) from the Marketplace or your repo
Runner	GitHub-hosted (`ubuntu-latest`, `windows-latest`, `macos-latest`, larger/ARM runners) or **self-hosted** (your machines, e.g. in your VPC; ARC runs them on Kubernetes)
Secrets / variables	Encrypted secrets and plain variables at repo, **environment** or organization level
Environment	Deployment target with protection rules (required reviewers, wait timer, branch restrictions) and its own secrets
Artifacts / cache	Files passed between jobs / dependency caches between runs

Syntax reference

```
name: ci                                    # shown in the Actions tab
run-name: CI for ${{ github.ref_name }}     # optional per-run title

on:
  push:
    branches: [main]
    tags: ["v*"]
    paths-ignore: ["docs/**", "*.md"]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      environment:
        description: Target environment
        type: choice
        options: [dev, staging]
        default: dev
  schedule:
    - cron: "0 3 * * 1"                     # Mondays 03:00 UTC

permissions:                                # least privilege for GITHUB_TOKEN (default for all jobs)
  contents: read

concurrency:                                # cancel older runs of the same branch
  group: ci-${{ github.ref }}
  cancel-in-progress: true

env:                                        # workflow-level environment variables
  PYTHON_VERSION: "3.12"

defaults:
  run:
    shell: bash

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    strategy:
      fail-fast: false
      matrix:
        python: ["3.11", "3.12"]
    services:                               # sidecar containers for tests
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s --health-timeout 3s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
          cache: pip
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q --junitxml=report-${{ matrix.python }}.xml
        env:
          DATABASE_URL: postgresql://postgres:test@localhost:5432/postgres
      - uses: actions/upload-artifact@v4
        if: always()                        # upload even if tests failed
        with:
```

```
          name: test-report-${{ matrix.python }}
          path: report-${{ matrix.python }}.xml
```

Keyword	Purpose
`needs: [test, lint]`	Wait for other jobs; access their outputs via `needs.<job>.outputs.<name>`
`if:`	Conditions: `github.ref == 'refs/heads/main'`, `github.event_name == 'push'`, `success()`, `failure()`, `always()`, `cancelled()`
`strategy.matrix`	Run a job for combinations (versions, OSes); `include` / `exclude`
`outputs:` + `$GITHUB_OUTPUT`	Pass values between steps/jobs: `echo "tag=v1" >> "$GITHUB_OUTPUT"`
`$GITHUB_ENV`, `$GITHUB_STEP_SUMMARY`, `$GITHUB_PATH`	Set env vars for later steps / write a Markdown summary / add to PATH
`environment: production`	Use environment protection rules and secrets
`continue-on-error`, `timeout-minutes`	Allow failure / prevent hung jobs
`working-directory`	Directory for `run` steps

Expressions and contexts

`${{ <expression> }}` evaluates contexts: `github` (event, ref, sha, actor, repository), `env`, `vars`, `secrets`, `matrix`, `steps.<id>.outputs`, `needs`, `inputs`, `runner`, `job`. Functions: `contains()`, `startsWith()`, `format()`, `toJSON()`, `fromJSON()`, `hashFiles('**/requirements.txt')`.

GITHUB_TOKEN and permissions

Every run gets an automatic short-lived `GITHUB_TOKEN`. Set **minimal** `permissions:` at workflow or job level (e.g. `contents: read`, `packages: write` to push to GHCR, `id-token: write` for OIDC, `pull-requests: write` to comment). Default organization settings should be read-only.

Secrets

- Add in Settings → Secrets and variables → Actions (repo / environment / org). Reference as `${{ secrets.NAME }}`; values are masked in logs.
- Secrets are **not** passed to workflows triggered by pull requests from forks.
- Prefer **OIDC** for cloud access instead of storing cloud keys:

```
permissions:
  id-token: write
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: arn:aws:iam::123456789012:role/github-deploy
      aws-region: ap-south-1
```

The IAM role's trust policy restricts which repo/branch/environment may assume it (see AWS notes, IAM roles).

Caching

`actions/setup-python|node|java|go` have built-in `cache:` options. General cache:

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: pip-${{ runner.os }}-${{ hashFiles('**/requirements*.txt') }}
    restore-keys: pip-${{ runner.os }}-
```

Docker layer caching: `docker/build-push-action` with `cache-from: type=gha` / `cache-to: type=gha,mode=max`.

Reusable workflows and composite actions

```
# .github/workflows/build-image.yml (in a central "platform" repo)
on:
  workflow_call:
    inputs:
      image-name:
        type: string
        required: true
    secrets:
      registry-token:
        required: false
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t "${{ inputs.image-name }}" .
```

```
# Caller in an application repo
jobs:
  image:
    uses: acme/platform/.github/workflows/build-image.yml@v1
    with:
      image-name: shop-api
```

Reusable workflows share whole jobs (golden-path pipelines across many repos). **Composite actions** (`action.yml` with `runs: using: composite`) bundle several steps into one reusable step.

Security hardening for Actions

Risk	Mitigation
Compromised third-party action	Pin actions to a full **commit SHA** (`uses: actions/checkout@<sha> # v4`), prefer verified creators, Dependabot for action updates
Over-privileged token	Minimal `permissions:`; org default read-only
Script injection	Never put untrusted input (`github.event.pull_request.title`, issue bodies, branch names) directly into `run:`; pass via `env:` and quote
`pull_request_target` / `workflow_run` misuse	These run with secrets in the base repo context: never check out and run untrusted PR code in them
Secrets exposure	Environment-scoped secrets, OIDC, no secrets printed, `::add-mask::` for dynamic values
Self-hosted runner compromise	Use ephemeral runners, never on public repos, isolate in their own network/accounts
Supply chain	Dependency review, CodeQL, artifact attestations / signed images

Lint workflows with **actionlint**; scan with zizmor/OpenSSF Scorecard.

Complete pipeline: test → build → scan → push → deploy (GitOps)

```
name: delivery
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: delivery-${{ github.ref }}
  cancel-in-progress: true

env:
  REGISTRY: 123456789012.dkr.ecr.ap-south-1.amazonaws.com
  IMAGE: shop/api

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: ruff check .
      - run: pytest -q

  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}

  image:
    needs: [test, security]
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write
    outputs:
      tag: ${{ steps.meta.outputs.tag }}
    steps:
      - uses: actions/checkout@v4
      - id: meta
        run: echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/github-ecr-push
          aws-region: ap-south-1
      - uses: aws-actions/amazon-ecr-login@v2
      - uses: docker/setup-buildx-action@v3
      - uses: docker/build-push-action@v6
        with:
          context: .
          load: true
```

```
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ steps.meta.outputs.tag }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ steps.meta.outputs.tag }}
          severity: CRITICAL,HIGH
          exit-code: "1"
          ignore-unfixed: true
      - run: docker push "${REGISTRY}/${IMAGE}:${{ steps.meta.outputs.tag }}"

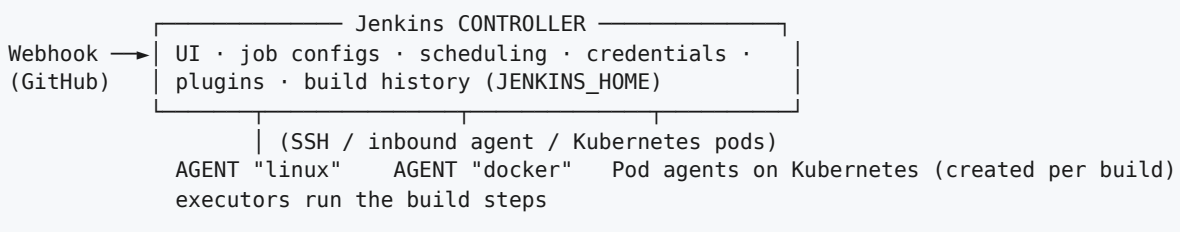
  deploy-dev:
    needs: image
    runs-on: ubuntu-latest
    environment: dev
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
        with:
          repository: acme/shop-config            # GitOps config repo
          token: ${{ secrets.CONFIG_REPO_TOKEN }}
      - name: Bump image tag for dev
        env:
          TAG: ${{ needs.image.outputs.tag }}
        run: |
          yq -i ".image.tag = \"${TAG}\"" apps/shop-api/values-dev.yaml
          git config user.name "ci-bot"
          git config user.email "ci-bot@users.noreply.github.com"
          git commit -am "shop-api: deploy ${TAG} to dev"
          git push
```

Argo CD watches the config repo and deploys the new tag (see GitOps notes). Production promotion is a pull request to `values-prod.yaml`, protected by review: an auditable, revertible release.

11.7 Jenkins in depth

Jenkins is the open-source, self-hosted automation server: extremely flexible through ~1,800 plugins, still very common in enterprises (often alongside newer tools).

Architecture



Term	Meaning
Controller	Central server: UI, configuration, scheduling. **Don't run builds on it** (set its executors to 0)
Agent	Machine/container executing builds; selected by **labels** (`linux` , `docker` , `arm64`)
Executor	A build slot on an agent
Job / project types	Freestyle (UI-configured, legacy), **Pipeline** (Jenkinsfile), **Multibranch Pipeline** (one pipeline per branch/PR automatically), Organization Folder (all repos of a GitHub org)

Term	Meaning
Plugins	Git, GitHub Branch Source, Pipeline, Credentials Binding, Docker Pipeline, Kubernetes, JUnit, Configuration as Code, Role-based Authorization
`JENKINS_HOME`	All configuration and build data: back it up

Installing (lab)

```
docker network create jenkins
docker run -d --name jenkins --network jenkins --restart unless-stopped \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts-jdk17
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open `http://localhost:8080`, unlock, install suggested plugins, create an admin user. Port 50000 is for inbound agents.

Declarative Jenkinsfile reference

```
pipeline {
  // where to run: agent any / none / docker {...} / kubernetes {...} / a label
  agent { label 'docker' }

  options {
    timeout(time: 30, unit: 'MINUTES')
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '30'))
    timestamps()
  }

  parameters {
    choice(name: 'TARGET_ENV', choices: ['dev', 'staging'], description: 'Deploy target')
    booleanParam(name: 'RUN_E2E', defaultValue: true, description: 'Run end-to-end tests')
  }

  triggers {
    pollSCM('H/15 * * * *')                          // prefer webhooks; polling as fallback
  }

  environment {
    REGISTRY = '123456789012.dkr.ecr.ap-south-1.amazonaws.com'
    IMAGE    = "${REGISTRY}/shop/api"
    TAG      = "sha-${env.GIT_COMMIT.take(7)}"
  }

  stages {
    stage('Checkout') {
      steps { checkout scm }
    }

    stage('Quality') {
      parallel {                                     // run in parallel
        stage('Unit tests') {
          steps {
            sh 'pip install -r requirements.txt -r requirements-dev.txt'
            sh 'pytest -q --junitxml=reports/junit.xml'
          }
          post { always { junit 'reports/junit.xml' } }
        }
        stage('Lint') {
          steps { sh 'ruff check .' }
        }
      }
    }

    stage('Build image') {
      steps { sh 'docker build -t "$IMAGE:$TAG" .' }
    }

    stage('Scan image') {
      steps { sh 'trivy image --severity HIGH,CRITICAL --exit-code 1 "$IMAGE:$TAG"' }
    }

    stage('Push') {
      when { branch 'main' }
      steps {
        withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
                          credentialsId: 'aws-ecr-push']]) {
          sh '''
            aws ecr get-login-password --region ap-south-1 \
              | docker login --username AWS --password-stdin "$REGISTRY"
            docker push "$IMAGE:$TAG"
          '''
```

```
        }
      }
    }

    stage('Deploy to dev') {
      when { branch 'main' }
      steps { sh './deploy.sh dev "$TAG"' }
    }

    stage('Approve production') {
      when { branch 'main' }
      steps {
        timeout(time: 1, unit: 'HOURS') {
          input message: "Deploy ${TAG} to production?", ok: 'Deploy', submitter: 'release-managers'
        }
      }
    }

    stage('Deploy to production') {
      when { branch 'main' }
      steps { sh './deploy.sh prod "$TAG"' }
    }
  }

  post {
    success { echo "Build ${env.BUILD_NUMBER} OK" }
    failure { echo 'Build failed - notify the team (Slack/email plugin)' }
    always  { cleanWs() }
  }
}
```

Section / directive	Purpose
`agent`	Where the pipeline/stage runs (`label` , `docker { image '...' }` , `kubernetes { yaml '...' }`)
`options`	Timeouts, concurrency, retention, timestamps, retries
`parameters`	Inputs for manual/parameterized builds (`params.TARGET_ENV`)
`triggers`	`cron` , `pollSCM` , upstream; webhooks via the GitHub plugin are preferred
`environment`	Variables (and `credentials('id')` bindings)
`stages` / `stage` / `steps`	Pipeline structure
`when`	Conditions: `branch` , `tag` , `changeset` , `expression { ... }` , `environment`
`parallel`	Run stages concurrently
`input`	Manual approval with allowed submitters
`post`	`always` , `success` , `failure` , `unstable` , `changed` , `cleanup`

Scripted pipelines (`node { ... }` , pure Groovy) are older and more flexible but harder to read; prefer declarative.

Credentials

Stored encrypted in Jenkins (or fetched from Vault/AWS Secrets Manager via plugins). Types: username/password, secret text, SSH key, secret file, certificate, AWS credentials. Use them only inside `withCredentials` / `credentials()` bindings; Jenkins masks them in logs. **Never** echo them or put them in the Jenkinsfile.

```
environment {
  SONAR_TOKEN = credentials('sonar-token')          // secret text → env var
}
steps {
  withCredentials([usernamePassword(credentialsId: 'dockerhub',
                                    usernameVariable: 'U', passwordVariable: 'P')]) {
    // single quotes: the shell expands $P, not Groovy (keeps the secret out of the command string)
    sh 'echo "$P" | docker login -u "$U" --password-stdin'
  }
}
```

Groovy string tip: use **single quotes** in `sh '...'` for secrets so Groovy doesn't interpolate them into the command string (which can leak them).

Shared libraries

Reusable Groovy code in a separate repo (`vars/buildPythonApp.groovy`), loaded with `@Library('acme-pipelines@v2') _`: teams then write a 3-line Jenkinsfile like `buildPythonApp(image: 'shop/api')`. This is Jenkins' version of golden-path pipelines.

Agents on Kubernetes

The Kubernetes plugin creates a **pod per build** with the containers you need (e.g. `python`, `docker` / `kaniko`, `kubectl`), then deletes it: elastic capacity, clean environments.

Operating Jenkins well

Area	Practice
Configuration as code	**JCasC** plugin (`jenkins.yaml`) + job DSL / multibranch; no manual click-configuration
Security	Disable builds on controller, enable CSRF protection, role-based authorization (Matrix/Role Strategy), SSO, least-privilege credentials, restrict script approvals
Updates	Keep core and plugins updated (plugins are the main source of vulnerabilities); test upgrades
Backups	Back up `JENKINS_HOME` (or rebuild from JCasC + Git)
Scale	Ephemeral agents (Kubernetes/cloud), build discarders, artifact storage outside Jenkins (S3/Artifactory)
Webhooks	GitHub → Jenkins `/github-webhook/` for instant builds

11.8 Other CI/CD platforms

GitLab CI

```
# .gitlab-ci.yml
stages: [test, build, deploy]

variables:
  IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA

test:
  stage: test
  image: python:3.12-slim
  script:
    - pip install -r requirements.txt -r requirements-dev.txt
    - pytest -q
  cache:
    key:
      files: [requirements.txt]
    paths: [.cache/pip]

build:
  stage: build
  image: docker:27
  services: [docker:27-dind]
  script:
    - echo "$CI_REGISTRY_PASSWORD"
      | docker login -u "$CI_REGISTRY_USER" --password-stdin "$CI_REGISTRY"
    - docker build -t "$IMAGE" .
    - docker push "$IMAGE"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy_prod:
  stage: deploy
  script: ./deploy.sh prod "$CI_COMMIT_SHORT_SHA"
  environment: production
  when: manual
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

AWS CodeBuild buildspec

```
version: 0.2
phases:
  install:
    runtime-versions:
      python: 3.12
  pre_build:
    commands:
      - pip install -r requirements.txt
      - aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY
  build:
    commands:
      - pytest -q
      - docker build -t $REGISTRY/shop/api:$CODEBUILD_RESOLVED_SOURCE_VERSION .
  post_build:
    commands:
      - docker push $REGISTRY/shop/api:$CODEBUILD_RESOLVED_SOURCE_VERSION
artifacts:
  files: [imagedefinitions.json]
```

(CodePipeline orchestrates source → CodeBuild → CodeDeploy/ECS deploy.)

Comparison

	GitHub Actions	Jenkins	GitLab CI	Azure DevOps	AWS CodePipeline
Hosting	SaaS (+ self-hosted runners)	Self-hosted	SaaS or self-hosted	SaaS or server	AWS managed
Config	YAML	Groovy Jenkinsfile	YAML	YAML / classic	JSON/console + buildspec YAML
Strength	Native to GitHub, huge marketplace	Ultimate flexibility, plugins, on-prem	All-in-one DevOps platform	Microsoft/.NET enterprises	AWS-native integration
Weakness	GitHub-centric	Maintenance and plugin sprawl	GitLab-centric	Complexity	AWS-only, less flexible

Kubernetes-native options: **Tekton** (CI pipelines as CRDs), **Argo Workflows**; CD: **Argo CD** / Flux (GitOps), Spinnaker.

11.9 Quality gates and testing in pipelines

Gate	Tool examples
Tests pass	pytest, JUnit, Jest; publish JUnit XML for reports
Coverage threshold	coverage.py / JaCoCo + `--cov-fail-under=80`
Code quality	SonarQube/SonarCloud quality gate, linters
Security	No critical SAST/SCA/image findings; no secrets
Performance	k6 thresholds (p95 < 300 ms) on staging
Manual approval	Production environments

Flaky tests (pass/fail randomly) destroy trust in CI: quarantine and fix them, never just re-run until green. Keep pipelines fast: run unit tests in parallel, cache dependencies, use test impact analysis, run long e2e suites on merge rather than every commit.

11.10 Securing the pipeline (DevSecOps in CI/CD)

Threat	Control
Leaked credentials in code/logs	Secret scanning + push protection, masked secrets, OIDC instead of static keys
Over-privileged pipelines	Least-privilege tokens/roles per job and environment; separate build and deploy permissions
Malicious dependencies / actions / plugins	Pin versions (SHAs/lock files), dependency review, SCA scanning, trusted registries/mirrors
Tampered artifacts	Sign images (cosign), generate provenance (SLSA), verify signatures at deploy (admission policies)
Poisoned pipeline execution (malicious PR changes the pipeline)	Protected branches, required reviews for workflow changes (CODEOWNERS on `.github/`), no secrets for fork PRs
Compromised runners	Ephemeral, isolated runners; no long-lived credentials on them

Threat	Control
Unreviewed production changes	Branch protection, required checks, environment approvals, audit logs

11.11 Troubleshooting pipelines

Symptom	Likely cause	Fix
Works locally, fails in CI	Different versions, missing env vars/secrets, OS differences, uncommitted files	Pin tool versions, reproduce in the same container image, check env
`permission denied` pushing image / deploying	Token/role missing permission, wrong OIDC trust condition, `permissions:` block too strict	Read the exact error; adjust least-privilege policy
`Error: Credentials could not be loaded` (OIDC)	Missing `id-token: write`, role trust policy `sub` doesn't match repo/branch/environment	Fix permissions and trust policy conditions
Tests pass/fail randomly	Flaky tests: timing, ordering, shared state, external dependencies	Isolate, mock, fix waits; quarantine
Pipeline very slow	No caching, serial jobs, huge Docker context, unnecessary e2e on every commit	Cache, parallelize, `.dockerignore`, path filters
Out of disk on runner	Large images/builds	Clean up, prune Docker, bigger runners
YAML error / workflow not triggering	Indentation, wrong event/branch/path filters, workflow on a different branch	actionlint, check `on:` filters and default branch
Secret empty in a PR workflow	PRs from forks don't receive secrets	Use `pull_request` safely; run privileged steps after merge
Deployment succeeded but app broken	No post-deploy verification	Smoke tests, health/metric checks, automated rollback
Jenkins build stuck "waiting for executor"	No agent with the label available, executors busy	Check labels/agents, add capacity

Approach: read the **first** error in the log (later errors are often consequences), re-run with debug logging (`ACTIONS_STEP_DEBUG=true` secret / Jenkins `set -x`), reproduce locally in the same container image, check what changed (dependency versions, secrets expiry, base images).

11.12 Labs

1. **CI for a Python app (GitHub Actions):** lint + pytest matrix on 2 Python versions, cache pip, upload test reports; protect `main` requiring the check.
2. **Container pipeline:** build the Docker image, scan with Trivy (fail on CRITICAL), push to GHCR with `sha-<commit>` tags using `GITHUB_TOKEN`.
3. **OIDC to AWS:** create an IAM OIDC provider + role trusted only for your repo's `main` branch; push to ECR without any stored AWS keys.
4. **Environments:** add `dev` (automatic) and `production` (required reviewer) environments; deploy jobs use environment-scoped secrets.
5. **Jenkins:** run Jenkins in Docker, create a Multibranch Pipeline from your repo, add the Jenkinsfile above (simplified), configure a GitHub webhook, store credentials, add an approval step.

6. **Break it on purpose:** introduce a failing test, a leaked fake secret (gitleaks should catch it), and a vulnerable base image (Trivy should fail): fix each.

11.13 Interview questions and answers

#	Question	Answer
1	CI vs continuous delivery vs continuous deployment?	Auto build/test on every change / always releasable with manual prod approval / automatic to prod
2	What stages would your pipeline have?	Checkout → lint → unit tests → security scans → build image → scan → push → deploy dev → integration tests → approval → prod → verify
3	What is "build once, deploy many"?	One immutable artifact promoted through environments; only config changes
4	How do you handle secrets in pipelines?	CI secret stores scoped per environment, OIDC for cloud, masking, no secrets in code/logs/images
5	Blue/green vs canary vs rolling?	Switch between two full environments / gradual traffic shift with metric checks / batch replacement
6	How do you roll back?	Redeploy previous artifact/digest (`helm rollback` , `kubectl rollout undo` , revert in GitOps repo), traffic switch for blue/green, feature flag off
7	How do you make pipelines fast?	Caching, parallel jobs, matrix, path filters, small images, test selection, fail-fast ordering
8	Artifacts vs cache?	Outputs you keep/pass on vs speed-up data safe to lose
9	How do you deploy DB schema changes safely?	Migration tools + expand/migrate/contract so each release is backward compatible
10	What are DORA metrics?	Deployment frequency, lead time, change failure rate, time to restore
11	GitHub Actions: job vs step vs action?	Job runs on a runner; steps inside run commands or actions; actions are reusable steps
12	How do you share pipeline logic across repos?	Reusable workflows/composite actions (Actions), shared libraries (Jenkins), templates/includes (GitLab)
13	How do you secure GitHub Actions?	Minimal permissions, pin actions by SHA, OIDC, environment protections, avoid untrusted input in `run` , careful with `pull_request_target`
14	Jenkins controller vs agent?	Controller schedules/configures; agents execute builds (no builds on the controller)
15	Declarative vs scripted Jenkins pipeline?	Structured opinionated syntax vs free-form Groovy
16	How do you store credentials in Jenkins?	Credentials store/Vault plugin, used via `withCredentials` / `credentials()` , single-quoted shell strings
17	A pipeline fails only in CI. Approach?	Read the first error, compare versions/env, reproduce in the CI image, check secrets/permissions, recent changes
18	What is a quality gate?	A required condition (tests, coverage, scans, approvals) a change must pass to progress
19	What is a feature flag and why use it?	Runtime toggle to enable features gradually; decouples deploy from release and allows instant disable
20	How do you verify a deployment?	Health checks, smoke tests, synthetic monitoring, error rate/latency analysis, automated rollback

#	Question	Answer
21	Self-hosted vs hosted runners?	Your machines (network access, custom tools, cost control, more security responsibility) vs managed, clean, ephemeral
22	What is GitOps-based delivery?	CI updates the desired version in a Git config repo; an in-cluster agent (Argo CD) applies it

11.14 Self-check: I can…

- [ ] Explain CI, continuous delivery and continuous deployment, and the core principles.
- [ ] Design a complete pipeline with the right stages, gates and test types.
- [ ] Explain artifacts vs caches, versioning, immutability, SBOMs and signing.
- [ ] Design environment promotion with config separation, approvals and safe DB migrations.
- [ ] Compare and choose deployment strategies and explain automated rollback.
- [ ] Write GitHub Actions workflows: triggers, matrix, services, caching, artifacts, outputs, environments, concurrency.
- [ ] Use OIDC to deploy to AWS without stored keys and harden workflows.
- [ ] Build reusable workflows/composite actions.
- [ ] Run Jenkins, write a declarative Jenkinsfile with parallel stages, credentials and approvals.
- [ ] Explain shared libraries, Kubernetes agents, JCasC and Jenkins security.
- [ ] Read GitLab CI and CodeBuild configurations.
- [ ] Measure delivery with DORA metrics.
- [ ] Secure the software supply chain in pipelines.
- [ ] Troubleshoot failing, slow and flaky pipelines.

REMEMBER

- **Code → Build → Test → Scan → Image → Registry → Deploy.**
- Prefer **OIDC** (short-lived credentials) over stored cloud keys.
- Never disable tests to make a pipeline green.